Tutorial HugoChatEditor

Un tutorial por Javidori Valsixseven Parrangas

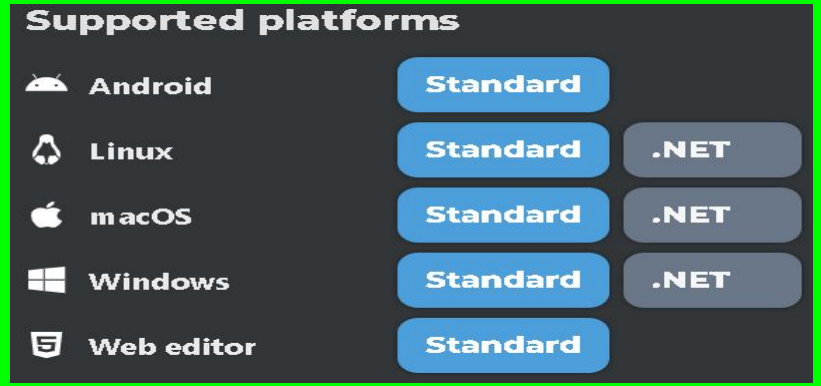
Primer paso - Tener la versión de Godot correspondiente

Primero que nada hay que tener la versión 4.6 se godot instalada, ya seas de Winslop o de Tux puedes bajar la versión correspondiente aquí:

https://godotengine.org/download/archive/4.6-stable/







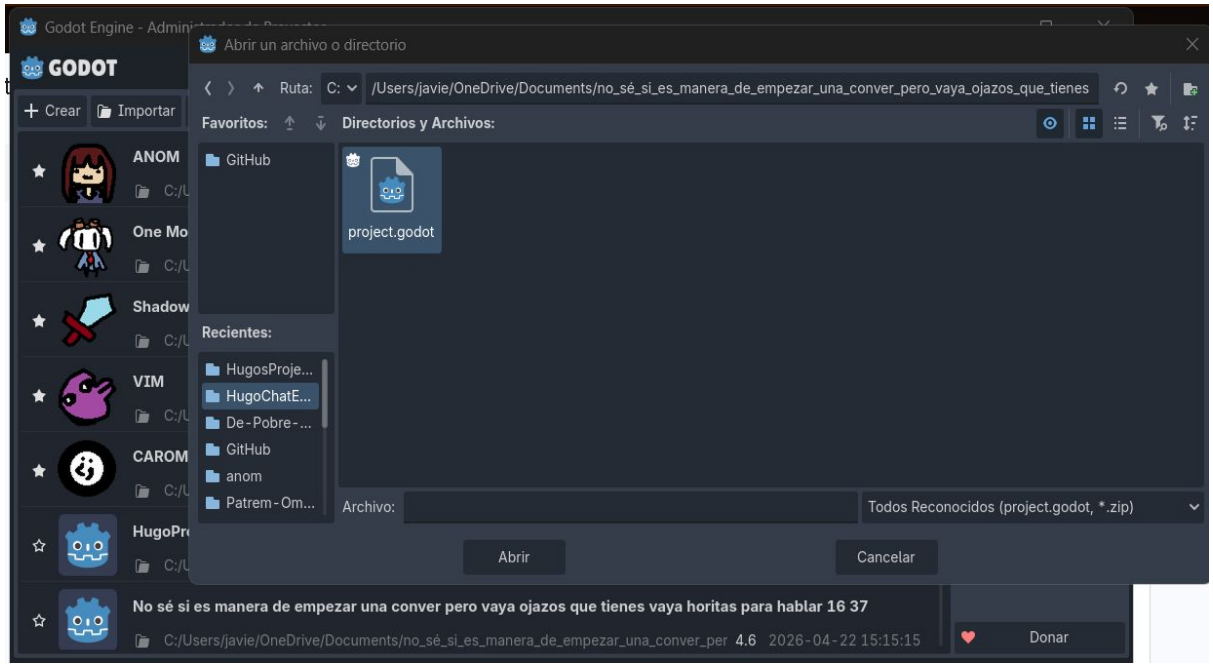
Segundo Paso - Importar Abrir el proyecto con Godot

Ejecutamos nuestro ejecutable de Godot y seleccionamos la opción de importar.

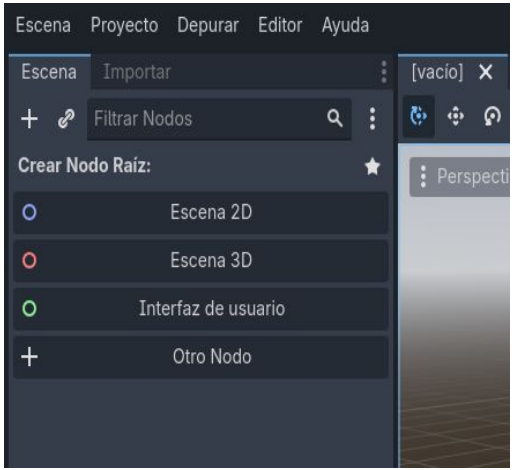
Elegimos el archivo

project.godot

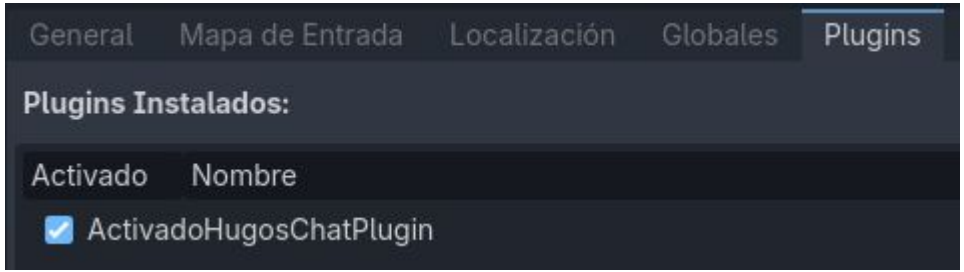




Tercer paso - Activar el plugin (si no está activado)



Vamos a Proyecto->Configuración del Proyecto->Plugins y Activamos el plugin en caso de estar desactivado.



Paso 6 y 7 - crear la raíz de escena

Por defecto el proyecto que abrirás tendrá una main_scene.tscn

que tendrá como raíz de escena el scene_tracker, scene_tracker

es la raíz de cualquier escena pues se encarga de llevar a cabo el proceso de lectura de entidades y componentes



Cuando quieras crear una escena asegúrate de que la raíz siempre sea este nodo.



Siguiente paso - Crear entidades y componentes

Podemos crear entidades y componentes al crear nodos, Entity para entidades sin transform y

Entity3D para Entidades con transform.

Para encontrar cualquier componente simplemente pon la palabra Component al crear

un nodo, por convención todos los componentes

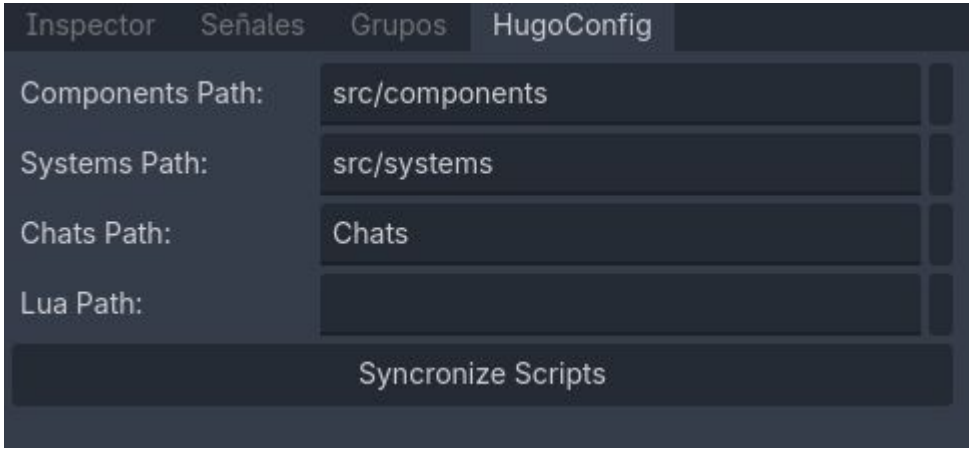
terminan en Component





Paso siguiente y medio + 1 - Configurar ruta de SYNC

Ya vendrán configuradas, pero en cualquier caso es importante saber que los scripts de componentes y sistemas que se hagan en c++ deben ser creados dentro de estas carpetas (hay soporte para subcarpetas). La ruta de Chats es donde se escribirán los .chat generados.



*Esta pestaña adicional aparece cuando el plugin está activado

Paso 5 - Guardar escenas

Al seleccionar el SCENE_TRACKER

que es la raíz de escena veremos

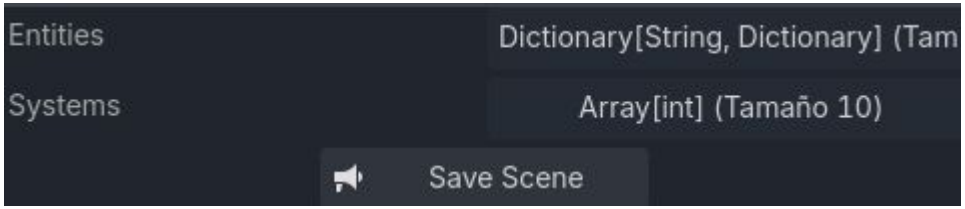
el botón Save Scene, son ese botón

por raro que parezca se puede

guardar la escena y se escribirá

en la ruta de los .chat configurada

anteriormente



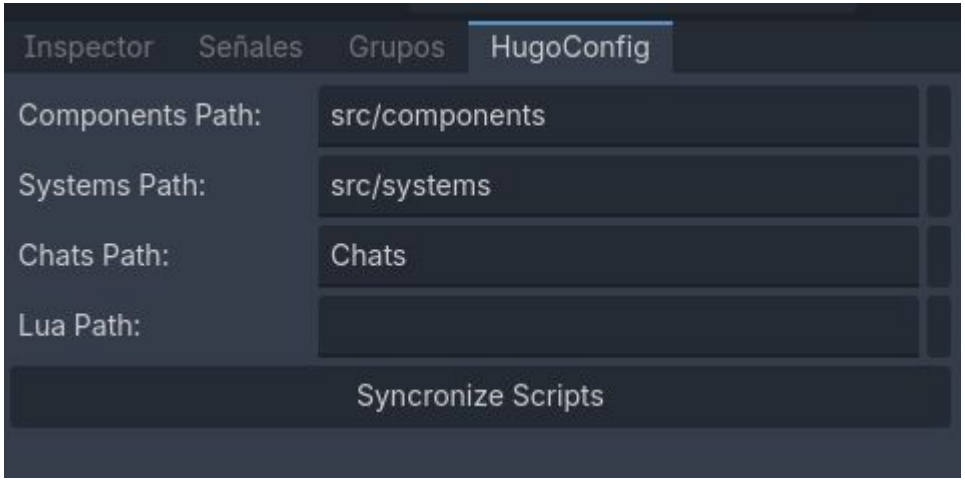


Paso 9 - Sincronizar scripts

Con el botón de Syncronize

Scripts podremos parsear los scripts escritos en c++ ubicados en la ruta de “Components Path:” a .gd

para poder usarlos con sus parámetros exportados en el editor.



**Nota importante: los scripts nativos del motor actualmente no se sincronizan si se hacen cambios en algún componente del motor es probable que se desactualice*

¿Qué hago si tengo que modificar un script nativo y usar su valor exportado?

No hay problema, se puede modificar fácilmente los análogos nativos para adaptarse a ello, los valores exportados simplemente tienen que seguir el patrón dado.

Se buscará a futuro una solución más elegante

```
@tool
extends Component
class_name AudioComponent

@export var type:int
@export var audioName:String
@export var volume:float
@export var loop:bool
@export var pitch:float
@export var startTime:float
@export var isPlaying:bool

func get_component() -> Dictionary[String, Variant]:
    return {
        "type" : FormatWriter.format(type, "type"),
        "audioName" : FormatWriter.format(audioName, "audioName"),
        "volume" : FormatWriter.format(volume, "volume"),
        "loop" : FormatWriter.format(loop, "loop"),
        "pitch" : FormatWriter.format(pitch, "pitch"),
        "startTime" : FormatWriter.format(startTime, "startTime"),
        "isPlaying" : FormatWriter.format(isPlaying, "isPlaying"),
    }
```